

Subject:	**Computer Organization and Architecture**
Topic:	**RISC vs. CISC — Choosing a Processor**
Class/Section:	**25BAI-601A**
Submitted by:	**Ridhi Gupta (25BAI10044)**
Submitted to:	**Dr. Ruchika Gupta**

1. Introduction

Processor architecture determines how a computer interprets and executes instructions. Two widely used instruction-set architectures are Reduced Instruction Set Computer (RISC) and Complex Instruction Set Computer (CISC). Although both perform computational operations, they differ in instruction complexity, decoding methods, addressing modes, and hardware implementation.

A technology company developing a battery-powered embedded device must select an architecture that provides efficient processing while minimizing energy consumption. This case study compares both architectures and identifies the more suitable option.

2. RISC and CISC Architectures

RISC (Reduced Instruction Set Computer): RISC uses a relatively small collection of simple instructions, generally designed to perform individual operations. It commonly follows a load-store approach, where data is transferred between memory and registers before arithmetic operations. Fixed-length instructions and simpler decoding make it suitable for many embedded applications. Examples include ARM and RISC-V.

CISC (Complex Instruction Set Computer): CISC provides a larger collection of instructions, including instructions capable of performing multiple operations. It generally supports variable-length instructions and several addressing modes. Although decoding can be more complicated, fewer instructions may be required for certain tasks. x86 and x86-64 are common examples.

3. Instruction Length and Decoding Complexity

Many RISC architectures use fixed-length instructions because they simplify instruction fetching and decoding. The processor can identify instruction boundaries easily, making instruction execution and pipelining more predictable. However, some RISC architectures, such as RISC-V, also support compressed instructions.

Instruction complexity directly affects decoding. RISC instructions generally have regular formats, allowing the control unit to interpret them efficiently. CISC instructions may have different lengths and complex operations, requiring additional decoding logic. Modern CISC processors often translate complex instructions into smaller internal operations called micro-operations (μops).

4. Addressing Modes and Example Instructions

RISC generally follows a load-store approach, meaning arithmetic operations are performed using registers rather than directly on memory operands.

RISC Example:

```
 LOAD R1, [1000]
LOAD R2, [1004]
ADD R3, R1, R2
STORE [1008], R3
```

The processor loads two values into registers, adds them, and stores the result.

CISC supports more addressing modes and can perform certain operations directly using memory operands.

CISC Example (x86-style):
MOV EAX, [1000]
ADD EAX, [1004]
MOV [1008], EAX
Here, the second value is accessed directly from memory during the addition, reducing the number of separate instructions.

5. Comparison Between RISC and CISC

The following table highlights the major differences between the two architectures:

Comparison Parameter	RISC Architecture	CISC Architecture
Instruction Set	Uses a relatively small set of simple instructions.	Provides a larger collection of instructions with varying complexity.
Instruction Length	Typically uses fixed-length instructions, although some extensions support variable lengths.	Generally, uses variable-length instructions.
Instruction Decoding	Simpler decoding because instructions follow more regular formats.	More complex decoding because instructions can differ in length and functionality.
Execution	Instructions are generally designed for simple and predictable execution.	Individual instructions may perform multiple operations and require different execution times.
Addressing Modes	Uses fewer addressing modes and generally follows a load-store approach.	Supports several addressing modes, including operations directly involving memory operands.
Registers	Typically provides a relatively large set of general-purpose registers.	Register availability and usage depend on the architecture.
Hardware Complexity	Generally, has simpler instructions, decoding and control logic	Generally, requires more complex instruction, decoding and control mechanisms.

6. Can CISC Outperform RISC?

Yes. A CISC processor can outperform a RISC processor depending on the workload and hardware implementation. Complex instructions can combine multiple operations, reducing the number of instructions required. Modern CISC processors also use pipelining, branch prediction, and out-of-order execution to improve performance. Therefore, instruction-set design alone does not determine processor speed.

7. Recommended Architecture for a Battery-Powered Device

For the proposed battery-powered embedded device, *RISC architecture is the preferred option*. The device primarily needs to collect sensor readings, perform calculations, and communicate data while operating with limited battery capacity.

A suitable RISC-based processor, such as an ARM Cortex-M microcontroller or a RISC-V microcontroller, offers several advantages:

- *Energy Efficiency:* Suitable implementations can operate with low power consumption, extending battery life.
- *Simpler Hardware:* Regular instruction formats simplify decoding and control operations.
- *Predictable Execution:* Useful for real-time sensor monitoring and control.
- *Cost Effectiveness:* Low-power microcontrollers are available at relatively affordable prices.
- *Embedded Support:* Integrated peripherals and communication interfaces support common embedded applications.

Although CISC processors can deliver excellent computational performance, their capabilities may be unnecessary for the relatively simple operations required by this device.

8. Conclusion

RISC and CISC offer different approaches to instruction execution. RISC emphasizes simpler instructions, predictable execution, and efficient hardware implementation, whereas CISC provides complex instructions and greater flexibility. For a battery-powered embedded device, a suitable RISC-based processor is the better choice because it can provide the required computational capabilities while supporting low-power operation. However, actual energy efficiency also depends on processor design, clock frequency, workload, and power-management features.

References:

- Patterson, D. A., and Hennessy, J. L., Computer Organization and Design: The Hardware/Software Interface.
- RISC-V International – Official Architecture Documentation.
- ARM – Cortex-M Processor Technical Documentation.
- Intel – Intel 64 and IA-32 Architectures Software Developer's Manual.